3 Query Optimization

3.1 Introduction

In this lesson, we shall utilize the database that we created during our introductory class to illustrate the concepts query optimization.

Query optimization is the difficult part of query processing. It determines **the efficient way to execute a query with different possible query plans**. It cannot be accessed directly by users once the queries are submitted to the database server or parsed by the parser. A query is passed to the **query optimizer** where optimization occurs.

The main aim of Query Optimization is to minimize the cost function, *I/O Cost + CPU Cost + Communication Cost*

It defines how an RDBMS can improve the performance of the query by **re-ordering the operations**. If the query is complex, query optimization helps in selecting the most efficient query evaluation plan from among various strategies.

3.2 Importance of Query Optimization

i. Query optimization provides **faster query processing.**
ii. It requires **less cost** per query.
iii. It gives **less stress** to the database.
iv. It provides **high performance** of the system.
v. It consumes **less memory**.

In high-level query languages, any given query can be processed in different ways. Resources required by each query will be different. DBMS has the responsibility to select the optimized way to process the query. Query optimizer uses relational algebra expressions to represent various way of expressing the same query.

3.3 Basic Steps in Query Optimization

The two basic steps involved in query optimization are:

- **Enumerating alternative plans** for evaluating the expression. Because number of alternative plans are large.
- **Estimating the cost of each** enumerated plan and choosing the plan with least estimated cost.

There are two methods of query optimization.

i. Cost based Optimization (Physical)
ii. Heuristic Optimization (Logical)

3.4 Cost based Optimization (Physical)

This is based on the cost of the query. The query can use different paths based on indexes, constraints, sorting methods etc. This method mainly uses the statistics like record size, number of records, number of records per block, number of blocks, table size, whether whole table fits in a block, organization of tables, uniqueness of column values, size of columns etc.

Methods such as dynamic programming, Left Deep Trees, and Interesting Sort Orders are used. Cost based optimizations are expensive but are suitable for queries on large data sets.

3.5 Heuristic Optimization (Logical) or Rule-based optimization

This method creates **relational tree** for the given query based on the equivalence rules. The **equivalence rules** provide alternative ways of writing and evaluating a query, from which the best path of evaluating a query can be selected.

The most important set of rules followed in this method is listed below:

I. ***Perform all the selection operations in the query as early*** *as possible. This reduces the number of records involved in the query, rather than using the whole tables throughout the query.*
II. ***Perform all the projections*** *as early as possible in the query. This is similar to selection but will reduce the number of columns in the query.*
III. ***Perform most restrictive joins and selection operations****. i.e. select those set of tables and views which will result in comparatively less number of records. Any query will have better performance when tables with few records are joined.*

Hence throughout heuristic method of optimization, the rules are formed to get lesser number of records at each stage, so that query performance is better. Consider the *university* database example that was hypothesized in our previous lesson.

3.5.1 Performing all the selection operations as early as possible

Suppose we have a query to retrieve *the students who are 18 years old and are taking BIT*. We can get all the student details from students table, and program details from the programs table. We can write this query in several different ways.

QUERY 1:

```
SELECT *
FROM students s, programs p
WHERE s.pid=p.id
AND s.age=18 AND p.pname='BIT';
```

QUERY 2:

```
SELECT * FROM
(SELECT * FROM students WHERE age=18) s,
(SELECT * FROM programs WHERE pname='BIT') p
WHERE s.pid=p.pid;
```

QUERY3:

```
SELECT * FROM (SELECT * FROM students WHERE age=18) s
JOIN (SELECT * FROM programs WHERE pname='BBIT') p
ON s.pid=p.pid;
```

Let us focus on the first two queries Query1 and Query2;

The two queries will return same result. However, there is a difference in how the result is processed. The first query will perform a cartesian product using the two tables, and then apply the filtering criteria to obtain the final result. By traversing entire tables to perform the cartesian product operation, an intermediate table with a large number of records is created.

The second query applies the filtering criteria on each tables first. This filtering operations reduces the number of records on each table (in programs table, the number of records is reduced to one in this case!). Then it performs cartesian product operation on these intermediary relations to obtain the final result. Thus, the cost in this case is comparatively less.

$$\sigma_{\texttt{s.pid=p.pid\^{}age=18\^{}pname='BIT'}}\ (\texttt{students} \times \texttt{programs})$$

☒ Not so efficient

$$\sigma_{\texttt{s.pid=p.pid}}\ (\sigma_{\texttt{age=18}}\ (\texttt{students})) \times (\sigma_{\texttt{age=18}}\ (\texttt{programs}))$$

☑ Efficient

The third query is even better. Just like the second query, it applies the filtering criteria on each tables first. However, the cartesian product is replaced by a join operation

$$(\sigma_{\texttt{age=18}}\ (\texttt{students})) \bowtie_{\texttt{s.pid=p.pid}} (\sigma_{\texttt{age=18}}\ (\texttt{programs}))$$

3.5.2 Performing all the projections as early as possible in the query

Consider a more specific output is required whereby only the name and programs details are required. Again, the query can be written in several different ways. Let's consider the two queries below;

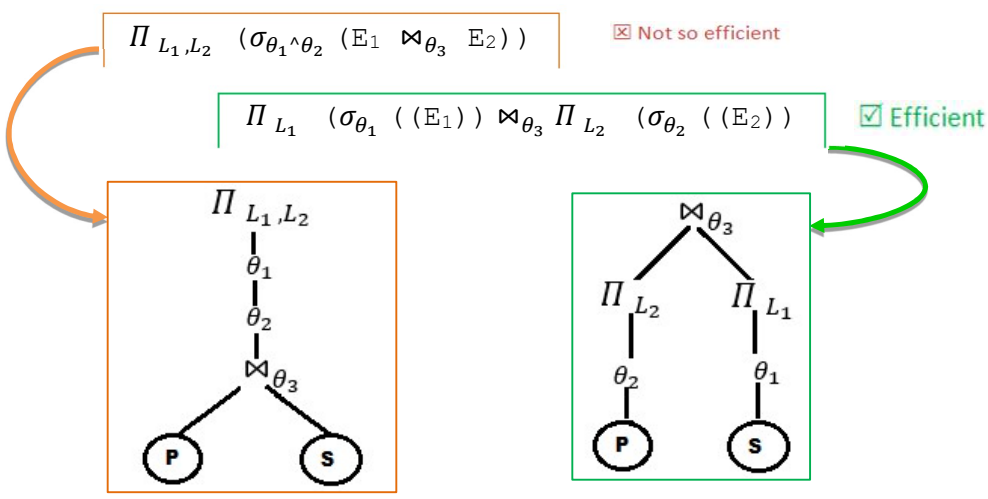
Query 1:

```
SELECT s.sname,p.pname FROM students s
JOIN programs p ON s.pid=p.pid
WHERE s.age=18 AND p.level='diploma';
```

Query 2:

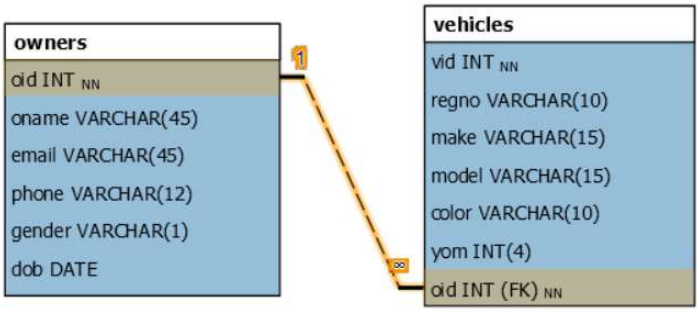
```
SELECT s.sname, p.program FROM
(SELECT sname, pid FROM students WHERE age=18) s
JOIN
(SELECT pname, pid FROM programs WHERE level='diploma') p
ON s.pid=p.pid;
```

Representing `students` using **E**$_1$, `programs` using **E**$_2$, `s.sname` using L_1, `p.pname` using L_2, `s.age=18` using θ_1, `p.level='diploma'` using θ_2, and `s.pid=p.pid` using θ_3 ; the equivalent expressions in relational algebra , and the equivalent representation in tree diagrams are as follows.



Assignment (To be submitted as instructed)

Consider the following ERD ;



(i) Write an SQL query to list the name, phone numbers of owners, registration number and make of vehicles that are red in color and have female owners.

a) Use Cartesian product operation.

b) Express the query in (a) using RA

c) Represent the query using a tree diagram

(ii) Rewrite the query in (i)above;

a) using a JOIN operation.

b) Express the query in (a) using RA

c) Represent the query using a tree diagram

(iii)Optimize the SQL statement in (ii) by;

a) performing projection and selection operations as early as possible

b) Express the query in (a) using RA

c) Represent the query using a tree diagram